

ORIENTATION — read this first

The entry point to ABRA. What the project is for, the plan, what each model does, the rules that govern the data, and the failure mode the project keeps hitting.

Rewritten 2026-09-24 for Reg M-C, in the Reg M-C 1.0.0 documentation pass. The previous edition described Reg M-B and a separate CHOMP that read ABRA's usage model; both are retired. It is in git at `commit 09bda887`, `docs/ORIENTATION.md`.

What this project is

ABRA builds a player for Pokémon Champions VGC, Reg M-C, with open team sheets, and takes it up the Showdown ladder (`gen9championsvgc2026regmcbo3`, account `medicham32`, cleared with Showdown staff).

The engine is the foundation; the search is the point. The plan has three steps:

- MEDICHAM is correct.** It is ABRA's own doubles simulator. The Reg M-C gate is OPEN, 10 of 10: the simulator agrees with Showdown on every board it compares, on three team lattices, on the damage stages, and on one staged scenario per legal item, ability and move (`docs/_reports/2026-09-24-regmc-gate-final.md`).
- So the solver searches on it, live, every turn.** At team preview, **CHOMP** solves the bring and the lead. Each turn, **XATU** holds the belief over hidden information, **MAG and DODUO** narrow the candidate joint actions, **MILTANK** fills a payoff matrix by playouts on MEDICHAM, **SLOWKING** solves it for an equilibrium mix, and **HYPNO** applies a capped exploit dial. **PORYGON2** is the value net that replaces long playouts, **MEW and MACHAMP** train it all by self-play, and **ROTOM** is the live client.
- It climbs the ladder.** The headline is a settled rating $\pm$ SD over many series and the per-series residual `S - E`. Never the peak.

The plan is `solver/PLAN.md`: the model registry (§2), the milestones Mo–M8 (§3), the live client (§4), the evaluation harness (§5). What has landed is `solver/LOG.md`. Every model earns its place by beating a named baseline on a test fixed before the run, or by failing honestly.

Reg M-B is retired (Will, 2026-09-24). Its record is closed at 7.0.0 in `CHANGELOG.md` and stays as history, labelled Reg M-B. No Reg M-B figure is compared with a Reg M-C one: they answer different questions.

The models

Every model except MEDICHAM is rebuilt from scratch under `solver/` and keeps its old name in a new role. The registry — inputs, outputs, dependencies, the test that proves each one and the baseline it must beat — is `solver/PLAN.md` §2. State on 2026-09-24:

Model	Role	State
MEDICHAM	the simulator, and its solver API (<code>engine/medicham_api.js</code>)	Reg M-C gate OPEN
CHOMP	team-preview solver: both open sheets → a mixed strategy over the bring/lead options	to build (M4)
XATU	belief over the opponent's back two and stat spreads	v1 built, store-only
MAG	per-slot action scorer: the human policy prior	v1 built, store-only
DODUO	joint coordinator: scores both slots as one joint action	v1 built, store-only

Model	Role	State
MILTANK	the search harness: candidates, payouts on MEDICHAM, the payoff matrix, the clock	v1 built; figures PRE-GATE, withheld
SLOWKING	the per-turn simultaneous-move solver	v1 built; unit-tested
PORYGON2	value net: state → P(win)	to build (M5)
GARY / HYPNO	human habits per situation / the capped exploit dial	to build (M7)
MEW / MACHAMP	self-play factory / training loop	to build (M5)
WOBBUFFET / DUSK	exploitability best-responder / endgame tables	to build (M6)
GURU	meta analysis	v0 built (descriptive), store-only
DITTO	team builder	to build (M8)
ROTOM	the live ladder client; replaces engine/mag_bot.js	to build (M3)
ALAKAZAM / KADABRA	the assembled agent / the coach	to build

The measured figures, with their artifacts, are in `MODELS.md`. The old Reg M-B implementations in `engine/` (`magnemite.js`, `miltank.js`, `mag_bot.js`, `pory.py`, ...) are history; each moves to a top-level `archive/` in the commit where its replacement passes its exit test.

The one principle that governs the data

Store raw, analyse on top.

Every replay is archived whole. The parsed store is a *derived* view. A new question is a **re-parse**, never a re-pull. Changing how games are segmented is a **re-filter**. Never design an analysis that requires re-fetching replays.

This has paid for itself more than once: measuring the format's true paralysis rate needed the `|cant|` lines the parsed store discards, and a reparse recovered games the parsed store had lost but the archive had kept.

The archive must stay complete or this principle is a lie. `MODE=backfill` refetches raw logs for any stored game missing from the archive, and `MODE=reparse` refuses to run while any stored game lacks one.

The second principle: know which games a number came from

`data/quality-filter.json` is the single definition of a usable game. `engine/quality.js` and `engine/quality.py` are thin readers of it.

The Reg M-C human dataset applies its own exclusions — named bots, accounts that behave like bots, custom-rule rooms, Illusion games, games with no result, and ABRA's own accounts — and prints the count for each (`docs/_reports/2026-09-23-human-dataset.md`). Two limits hold for any of these sets:

- **Bot detection is a floor, not a proof.** Call the surviving set "no bot detected", never "human".
- **A statistic that needs a full bring is conditioned on games long enough to show one.**

The funnel below is regenerated by CI over `data/games.ladder.json1`, the Reg M-B closed-sheet ladder store `engine/quality.js` reads by default. It is kept because the filter it demonstrates is the same filter, not because that store is the target:

<!-- BEGIN:FUNNEL --> Of **94,360** games collected, **26,134** are usable — **27.7%**.

Games are dropped for five reasons, in this order:

Stage	Games remaining
collected	94,360
after removing named bots	55,256
after removing accounts that behave like bots	49,215
after removing forfeits	32,912
after removing games under 3 turns	32,677
after requiring all four brought to be revealed	26,134

<!-- END:FUNNEL -->

Why the bot filter is not paranoia. On Reg M-B a small number of undetected bot accounts played the same six Pokémon across a large share of the store, and the pre-filter top of the usage table *was* that team. That is why every tool must go through the filter:

<!-- BEGIN:RAWREADERS --> **19 engine tools still read the store with neither the clean filter nor a declared reason.** `engine/selftest.js` fails while any remain, and names them:

`engine/argmax_paired.js` , `engine/bench_speed Consolidate.js` , `engine/calibrate.py` ,
`engine/click_census.js` , `engine/coach.js` , `engine/derive_sets.js` , `engine/feature_engine_contrast.js` ,
`engine/forced_switch_audit.js` , `engine/medicham2-browser.js` , `engine/mew_farm.js` ,
`engine/next_regulation_ingest.js` , `engine/rollout_r1_join.py` , `engine/stamp.js` ,
`engine/validate_store.js` , `tests/test-medicham-coverage.js` , `tests/test-next-regulation.js` , `tests/test-parse.js` , `tests/test-side-guard-chooser.js` , `tests/test-workflow-paths.js`

Anything they publish is computed over a store that is 72.3% unusable. <!-- END:RAWREADERS -->

The engineering standards, and why each exists

Every standard in `docs/ARCHITECTURE.md` was written after a specific failure. They are not preferences.

- **S1 — single source of truth.** Where several representations are unavoidable, one is definitive and the rest are **generated by a script**, never hand-synchronised.
- **S2 — duplication that cannot be removed must be observable.** A contract test asserts every implementation agrees.
- **S7 — the store has a shape, and it is tested.** No duplicate ids; `brought ⊆ six` ; `lead ⊆ brought` ; the winner is one of the two players.
- **S8 — measured, never asserted.** No constant that affects a result is typed by hand.
- **S9 — golden master before refactoring.** Record the outputs, change the code, compare.
- **S10 — enumerate closed domains.** Walk the whole domain and assert direction. A spot-check cannot detect a missing row, because a missing row usually degrades to a plausible default.
- **S11 — one publisher.** Exactly one process commits and pushes.

The failure mode this project keeps hitting

Silent wrongness. Not crashes — the code returns a number, and the number is wrong. The shapes:

- A table that was **incomplete** rather than incorrect, so counting it looked fine.
- A check **aimed away from where faults occur.**
- A **plausible causal story** that nobody verified, which then propagated into several documents.
- A number **retracted in one place** but left standing elsewhere.

- **A capability that was absent while everything reported success.** Every capability now emits a counter, and a zero is called out.

The countermeasure is not care. It is: assert direction rather than count, aim checks where faults occur, treat causal claims as hypotheses until tested, make a retraction fail a build, and print state rather than type it.

Where the rules come from

Champions is a real Showdown format with its own mod, and **the mod is the authority**. For Reg M-C it is the `pokemon-showdown-mc` checkout, selected by `ABRA_REGULATION=regmc`. Where MEDICHAM and the authority disagree, the authority is right, and the gate measures the disagreement. No Pokémon value is typed from memory: derive it from the format, filtered to the regulation, or cite the line it was read on.

Stat convention: Champions uses **SP**, not EVs — a 66-point budget, 32 per stat.

Who does what

Five divisions, cut on what a change invalidates (`DIVISIONS.md`):

Division	Owns
ENGINE	MEDICHAM: the simulator, the tags, the differential
MEASURE	whether a number is true: provenance, staleness, SPRT reading
SOLVER	everything under <code>solver/</code> — search, the nets and the live client (was SEARCH until 2026-09-24)
OPS	ingest and the stores (read-only)
WEB	the site; renders numbers, never authors one

Cowork drafts prose into `docs/_inbox/` and never authors a number or runs git. Claude Code measures, tests and is the only publisher.

How to read the rest of the docs

Document	Purpose
<code>solver/PLAN.md</code>	The plan: model registry, milestones, live client, evaluation
<code>solver/LOG.md</code>	What landed, newest first
<code>SUMMARY.md</code>	Whole-project summary, one table per component
<code>MODELS.md</code>	Per-model ledger; Reg M-C models first, Reg M-B entries archived below
<code>REGMC.md</code>	The Reg M-C line: the gate, the version scheme
<code>REGULATION-ROTATION.md</code>	What has to change when a regulation rotates
<code>ABRA-whitepaper.md</code> , <code>ABRA-deck-plain-english.md</code> , <code>ABRA-technical-docs.md</code>	Technical, plain-English, and ASD-STE100 editions
<code>DIVISIONS.md</code> , <code>{ENGINE,MEASURE,SOLVER,OPS,WEB}.md</code>	Who owns what, and each division's working ledger
<code>RUNNING-NOTES.md</code>	One row per change between majors
<code>CHANGELOG-REGMC.md</code> / <code>CHANGELOG.md</code>	The Reg M-C line / the closed Reg M-B record

State is printed, never typed:

```
node engine/status.js  
ABRA_REGULATION=regmc node engine/quarantine.js  
node engine/open_work.js
```

Where a document disagrees with a measurement, the measurement wins.

Honest limitations

- **No strength claim yet.** Every arena figure measured so far is PRE-GATE and is withheld until it is re-run on the release the Reg M-C gate opened on.
- **Open team sheets only.** Closed sheets and bo1 are out of scope.
- **Illusion is the one declared exclusion** from the simulator's gate.
- **The store holds no stat spreads**, so XATU narrows them only from what a game reveals.
- **The ladder has not been played.** ROTOM is not built, and the ladder can only resolve differences of tens of rating points over hundreds of series (`solver/PLAN.md` §5).